Source-Constrained Information Flow for Shared Agent Memory

OpenBot

***Abstract*—Shared agent memory can join sources that have different readers. We study when a compact access label keeps every source rule through later derivation. We give a necessary and sufficient condition for safe binary label composition, then apply it to a baseline whose group tokens also depend on contributor tags. The baseline violates that condition: a contributor union can widen a token's meaning. Exhaustive enumeration of 15,376 source pairs and eight viewers finds 1,216 unsafe admissions and 352 lost permitted reads. Source-specific predicate conjunction is the largest reader set that preserves every input rule; a graph proof states the assumptions needed to use it. We also define complete input capture, checked release, and a version barrier for removal. Finite graph checks support the model, but do not establish language-model confidentiality. A separate paired attack and utility study remains unrun.**

***Index Terms*—Agent memory, Information flow, Source policy, Provenance**

I. Introduction

A shared agent can help people in several organizations and in private or group chats. Memory makes earlier work useful in later tasks. It also creates a question that ordinary access checks do not answer: who may learn an observation made from several private sources?

Suppose Ana gives an agent a private work plan. Bob gives it a private delivery date. The agent can infer which project will be late. A filter that hides the original records may still show that inference. The inference can reveal either source without repeating its wording. The relevant boundary is the information on which an output depends.

This paper studies that boundary. It does not study whether memory learns sound beliefs or whether an agent may perform an outside action. Its question is which readers may receive knowledge derived from sources with distinct access rules. We use information-flow control, which follows limits on reading as information moves through computation.

This paper combines formal analysis with finite experiments. Its contributions are: a testable condition for safe composition of context-dependent labels; a small counterexample and exhaustive count of the ways a token baseline fails that condition; and source-graph rules with explicit proof and evaluation boundaries. The finite counts describe the stated policy domain, not how often users face a failure. A language-model attack study is a separate part of the research plan.

The method applies existing information-flow ideas; it does not claim a new label lattice. Its focus is memory records that serve both as evidence for later records and as model input. Model-declared citations need not name every influence. Also, source metadata can change an access token's meaning. Both issues matter before source visibility can constrain derived knowledge.

II. Relation to prior work

Denning's lattice model gives a formal order for permitted information flow and combines security classes when inputs are used together [1]. Our reader-set model uses the same central idea. A smaller allowed reader set is a stronger restriction. Adding a source therefore cannot enlarge the permitted set without a separate release step.

Myers and Liskov's decentralized model makes this relation more direct. Labels can contain several owners, each with a reader policy. Combining labeled inputs keeps the owners and restricts effective readers [2]. Their later paper gives sound and complete rules for secure relabeling [3]. We specialize this approach to source records, organizations, and group chats. Those results concern their stated label semantics. The memory predicates and model boundaries defined below still require their own argument.

Goguen and Meseguer define noninterference in terms of what one set of users can observe about another set's actions [4]. This is stronger than returning only rows that pass a predicate. A denied source may still affect generated text, result order, version numbers, or response time. Our proofs name their observation boundary. They do not silently extend a record-filter property to all effects of a model call.

Declassification is an explicit permission to release information that a normal flow rule would keep private. Sabelfeld and Sands organize declassification by what is released, who may release it, where release occurs, and when it is allowed [5]. Zdancewic and Myers study robustness: an untrusted input must not gain control over trusted release decisions [6]. These ideas matter when a model proposes that a memory entry be shown to more people. A reason written in natural language is useful evidence, but it is not proof of the right to release.

Buneman, Khanna, and Tan distinguish forms of data provenance, the record of where data came from [7]. Green, Karvounarakis, and Tannen give an algebra for tracking database provenance [8]. W3C PROV supplies a standard model for entities, activities, agents, and derivation [9]. We use source links to support audits and removal. Cheney, Ahmed, and Acar connect provenance to a formal notion of dependency correctness, with sound approximations in a defined query language [10]. Such links do not by themselves impose confidentiality. Nor do citations chosen by a language model establish a complete account of influence. This distinction between stated evidence and possible influence is central to the method.

Costa et al. apply information-flow control to agent planners in Fides [11]. They track both confidentiality and integrity, and separate integrity noninterference from the weaker confidentiality property of explicit secrecy. Their reader-set example already uses intersection. Our contribution is not that rule, nor a general agent-flow system. We test whether a compact memory label still means the same thing after source metadata is combined. We also keep derivation and removal obligations for long-lived records explicit. Fides is a required system comparison for a future model study, not a baseline we have run here.

AgentPoison studies attacks that place harmful examples in agent memory or a knowledge base [12]. MINJA weakens the required access: an attacker can insert useful attack records through ordinary queries and observed replies [13]. These works concern memory integrity and attack behavior. Our property concerns confidentiality after derivation. A record can be true yet have an unsafe reader set; it can also have a safe reader set and contain false or hostile text. Neither property implies the other. These attacks motivate hostile-text fixtures, but their published success rates are not measurements of our access model.

III. Defined audience-and-contributor baseline

We define a baseline that stores one shared pool, possibly used from several organizations. This is an abstract design, with no deployment claim. The storage location does not grant read access. The trusted controller supplies a viewer $v = (o, k, x)$: organization o, context k, and person or group identifier x. The contexts private and group are disjoint. Document text cannot change it.

A record e contains text, topic tags, a source list, a state, allowed organizations O_e, contributors C_e, origin keys Q_e, and audience tokens A_e. Origin keys name source chats, groups, or tools. Topics are ordinary model-chosen words. Only the controller writes the policy fields. This separates descriptive text from access rules but does not yet prove that the rules compose safely.

Audience tokens have three forms: U means all users of the shared agent; $P(p)$ means person p; and $G(g)$ means group g. Define the token meet m by

$$m(U, a) = m(a, U) = \{a\}, \tag{1}$$

$$m(a, a) = \{a\}, \tag{2}$$

with $m(a, b) = \varnothing$ for unequal tokens when neither is U. For token sets, $M(A, B)$ is the union of $m(a, b)$ over all pairs $a \in A$ and $b \in B$. Combining a nonempty list of sources unions their contributor and origin sets, intersects their organization sets, and folds M over their audience sets. Repeated sources have no extra effect. An empty source list is rejected for a derived record.

Visibility is defined by the following equations. Square brackets denote the truth of a condition. The function F gives the meaning of one token in its record's contributor context.

$$V_e(o, k, x) = [o \in O_e] \wedge (\exists a \in A_e : F(a, k, x, C_e)), \tag{3}$$

$$F(U, k, x, C) = \text{true}, \tag{4}$$

$$F(P(p), k, x, C) = [k = \text{private}] \wedge [x = p], \tag{5}$$

$$F(G(g), k, x, C) = ([k = \text{group}] \wedge [x = g]) \tag{6}$$

$$\vee\, ([k = \text{private}] \wedge [x \in C]). \tag{7}$$

Thus a private contributor can read a group entry outside the group. A group answer cannot use a personal entry merely because that person spoke. No matching organization or no audience token means denial. The model contains no general membership relation. In particular, a person token and a group token have empty token meet even when the person might belong to that group.

The baseline admits a learned record when its named sources exist and are active. The controller computes its fields with the combination rule. The model selects the source IDs and content. Admission does not prove that the text depends only on those IDs. A direct private input instead creates a root with its person's audience. Roots record original evidence; derived records point to earlier records.

For later work, the baseline sends only active records satisfying $V_e(v)$ to an answer model. Any returned citation must name an admitted record. An optional broader helper reads the pool and returns text under prompt instructions alone. A proposed audience expansion adds a token and saves a reason string, without checking a separate authority object. These are explicit baseline choices that the analysis will test, not parts of the corrected method.

Invalidation marks only the selected record inactive. Baseline removal deletes records whose contributor, origin, or organization fields contain a supplied key. A global version increases after every pool change. These definitions make later failure cases reproducible: neither descendant invalidation nor metadata secrecy follows automatically from the visibility equations.

IV. A source-constrained model

A. Threat and observation boundary

The attacker may submit text in one permitted chat, control text from an outside tool, choose misleading citations, and propose false consent. It may later query through another account whose rights it holds. It knows the algorithm. Its goal is to make an answer or saved record reveal a source that the receiving viewer may not read. The attacker may cause model output to be arbitrary; the record theorem does not trust model obedience.

The trusted part consists of viewer authentication, the policy interpreter, source IDs and versions, the input recorder, and the store transaction that admits or blocks a record. An attacker cannot write those fields directly, replace controller code, or bypass storage checks. A compromised controller is outside the model. An outside model that retains private data between calls is also outside the output theorem below. This is a material trust boundary, not a property established by a label check.

The primary observable is an admitted record's text and identity. The stronger output claim also needs its stated public transcript. Timing, resource use, errors, search order, counts,

and provider logs are excluded unless a deployment adds them to that transcript and proves that they depend only on permitted input. Denial is not automatically harmless: whether a record exists or whether an error occurs can itself carry private information.

B. Records, viewers, and policies

The proposed method retains the baseline viewer context. A group answer is visible to the group, not just to the person who spoke. The controller authenticates that context before memory selection. The threat model allows hostile text, false citations, and misleading release requests, but assumes the controller authenticates viewers and preserves policy fields.

Let $S(e)$ be the direct sources of record e, and let $R(e)$ be its source roots. A root is an original evidence item or a direct private-input event. Each root has a unique event ID and a policy, even when it has no earlier sources. This avoids confusing a source-free public fact with a direct private input. Source references name immutable versions, so editing a source does not change an earlier derivation silently.

Each root s has a predicate $P_s(v)$: it is true exactly when that viewer may receive the source under the selected policy. The predicate includes organization and audience context. It can preserve the baseline participant rule, but its participant set belongs to that particular source. It must not read the combined contributor set of a derived entry.

For an ordinary derivation with sources $e_1, ..., e_n$, the rules are:

$$R(e) = \cup_{i=1}^{n} R(e_i), \quad n > 0. \tag{8}$$

$$P_e(v) = \wedge_{i=1}^{n} P_{e_i}(v). \tag{9}$$

Here union retains every root, and logical conjunction requires every source policy. This is the familiar information-flow join expressed as an intersection of readers. Origins and allowed organizations must be kept separately: origin organizations accumulate by union, while allowed organization sets intersect. The former answers where removal must follow. The latter answers where reading may occur.

An empty intersection is meaningful. The agent may store a derived entry that no ordinary viewer may read. It may remain useful to an authorized maintenance process. A system may instead refuse to store it to reduce retention. Neither choice permits an automatic widening merely because an empty audience has low utility.

C. Complete input tracking

A source list can mean two different things. Evidence sources explain why a statement seems true. Influence sources describe inputs that may have affected its form or content. For confidentiality, the second meaning is required. A model can cite a public item while copying a detail from an uncited private item in the same prompt. Checking that cited IDs exist does not detect this case.

The proposed conservative rule records every protected input supplied to a model call. Generated text inherits their conjunction, even if the model reports fewer supporting IDs. This includes protected topic lists, summaries, tool data, and earlier messages. System prompts and model state must either be common public inputs or have their own policy. A model's self-report cannot remove a dependency.

Input capture must also cover control choices. If a private bit selects one of two public prompts, recording only the chosen prompt loses that bit's influence. The same issue arises when a hidden record changes retrieval order, the number of model calls, or which public item is selected. The host must either label these choices with the inputs that control them, or make selection a function of the target viewer's permitted slice. Tracking only message bodies is insufficient.

This rule has a clear utility cost. A helper given every person's private entries can produce an entry with no common reader. To recover useful sharing, generation must receive narrower input. One option runs the helper on a target viewer's permitted slice. Another groups calls by equal source policies. If the same text is independently produced from a less restricted input set, it becomes a separate record with its own derivation. Text equality alone is not enough to replace the older record's sources.

The proposed store keeps a source graph and a conservative policy cache. A graph edge says one record used another. Cached decisions include source and policy versions. A read may use the cache only while those versions remain current. Missing roots, an unknown membership answer, or a stale policy cache cause denial until they can be checked.

Proposed admission and read rules

1)	Capture all protected inputs before a model call.
2)	Form the source union and policy conjunction.
3)	Validate source versions when saving the output.
4)	Store evidence citations apart from influence sources.
5)	At read time, check the viewer and every live constraint.
6)	For broader release, require a separate release record.

Fig. 1. The model can propose content. The host owns the source and policy record.

D. Group knowledge and uncertainty

A channel ID can stand for a shared reading context without supplying a full membership list. In that case, matching channel context is sufficient for a channel-scoped entry, but a claimed private membership is not. A person-channel meet can be empty in the token representation even when the person belongs to that group. This loses some permitted reuse. It is a conservative result of missing evidence, not proof that the person is outside the group.

A deployment that wants membership-based private access must choose its policy. Membership when the source was created differs from membership now. A new member may have current channel access but no right to old private history. A former member may retain knowledge already seen but lose the right to future retrieval. The policy must name the time rule and trusted membership provider. On provider failure, an uncertain private membership must not become a positive answer. The baseline deliberately leaves this relation undefined.

V. Analytical claims and a concrete limit

A. A criterion for safe label composition

Let L be a set of stored labels. At a fixed policy version, $D(l)$ is the set of viewers admitted by label l. It includes every field used by the reader check, such as contributor context. Let $J(l_1, l_2)$ be the host's binary combination rule. Assume the output label is in L and later derivations use the same rule. A tree may use any label in L as a leaf, including a saved derived label. A valid tree uses only combinations that the host admits.

Theorem 1 (local composition). Every finite binary derivation tree admits only viewers who may read each leaf if and only if

$$D(J(l_1, l_2)) \subseteq D(l_1) \cap D(l_2) \tag{10}$$

for all labels that can occur together at a derivation step. The quantifier must cover intermediate labels, not only input roots.

Proof. Necessity follows from the tree with exactly those two leaves. For sufficiency, a leaf trivially preserves its own reader set. At an internal node, the displayed inclusion requires every admitted viewer to read both children. Applying the induction hypothesis to each child gives every leaf below it. This proof needs neither associative labels nor a balanced tree. It needs a fixed interpretation of each stored label at the decision being proved.

The equality case also gives a useful maximality statement. If an ordinary derived value may depend on all its inputs, any reader set that preserves each input rule is contained in their intersection. Choosing the intersection therefore loses no reader allowed by this conservative contract. This is not a claim that every output needs every input: a constant output may have no semantic dependence. Removing an input constraint requires proof of that fact or a checked release; the model's choice of citations is neither.

If a token set A is interpreted using context c, write its readers as $D(A, c)$. A token meet is not enough: the real proof obligation is

$$D(M(A_1, A_2), f(c_1, c_2)) \tag{11}$$

$$\subseteq D(A_1, c_1) \cap D(A_2, c_2), \tag{12}$$

where f combines all context that changes the token's meaning. A meet proved for fixed token meanings does not imply this condition when f changes contributors or membership. The next example violates exactly this obligation.

B. Properties of the defined baseline

Organization preservation. For a nonempty combination, every member of the result's allowed organization set belongs to every source's allowed organization set. This follows directly from repeated set intersection. An audience expansion leaves that set unchanged, so it does not by itself make a record visible in another organization. This is a record-level claim; it assumes the viewer's organization is correct.

Origin preservation. Every contributor and origin key of a source appears on the combined record. Repeating the combination preserves these sets through a chain by induction. This assumes later operations do not remove their elements. It does not establish preservation of origin organizations, because the baseline uses only the intersected organization set.

Restricted visibility preservation. Fix a group viewer and a nonempty finite source list combined by the baseline rules, with no audience expansion. If the viewer passes the combined filter, it passes every source filter. The group branch accepts only a matching channel or the all-user token. The token meet can produce such a token only if each source contains an accepting token. Organization intersection supplies the other part of the claim. It also works for a private viewer when the accepting token is U or $P(p)$. Thus having no group tokens in any source is a sufficient condition for private-view preservation, though it is not necessary.

The private participant branch needs a separate analysis.

C. Contributor union changes audience meaning

Take sources a and b with $O_a = O_b = \{o\}$, $A_a = A_b = \{G(g)\}$, $C_a = \{\text{Ana}\}$, and $C_b = \{\text{Bob}\}$. Let $v = (o, \text{private}, \text{Ana})$. The private branch of F gives $V_a(v) = \text{true}$ because Ana belongs to C_a, and $V_b(v) = \text{false}$ because she does not belong to C_b.

For their derived record d, the baseline gives $O_d = \{o\}$, $A_d = \{G(g)\}$, and $C_d = \{\text{Ana}, \text{Bob}\}$. Therefore $V_d(v) = \text{true}$. The desired implication from derivative visibility to every source's visibility is false: its premise is true and its second source conclusion is false.

TABLE I
All rows share organization o and group audience g. Contributor union changes the meaning of the common audience token.

Record	Contributors	Ana privately
Source a	Ana	Allow
Source b	Bob	Deny
Derived d	Ana, Bob	Allow

This counterexample follows from the defined equations. It does not assume that Ana never saw Bob's words in a real group. It compares the permitted private views of the sources and derivative. A different policy that grants all group members private access to all group history would need a different source predicate too.

The proposed conjunction gives $P_d(v) = P_a(v) \wedge P_b(v) = \text{false}$. For a viewer in group g, both predicates are true, so the derivative remains available there. Retaining the source-specific contributor sets repairs this case without denying the legitimate group read. The counterexample follows without a model run. The finite enumeration below compares separate implementations of its rules.

D. Why the aggregate fields cannot give an exact repair

Proposition (lost source meaning). No reader check that sees only the baseline's combined organizations, contributors, and audience tokens can always be both safe and complete for source-reader conjunction.

Proof by two traces. Keep one organization and group g. In trace one, combine $a = (\{o\}, \{\text{Ana}\}, \{G(g)\})$ and $b = (\{o\}, \{\text{Bob}\}, \{G(g)\})$. In trace two, keep a but replace b's

audience with $\{U\}$. Both combinations have organization $\{o\}$, contributor set $\{\mathrm{Ana}, \mathrm{Bob}\}$, and audience $\{G(g)\}$. Ana may read both sources only in trace two. A check of the identical combined fields must return the same answer for both. Allow is unsafe in trace one; deny loses a permitted read in trace two.

The two traces use only one contributor per source. The lost distinction is whether Bob's source admits Ana on its own. Retaining source references and following their policies can recover it; recomputing from the three aggregate fields cannot. This does not require storing full viewer sets. A list of source policy clauses also keeps the distinction. Its cost depends on the number of distinct clauses, while its reader check remains a conjunction.

E. A bounded theorem for the proposed method

Theorem 2 (source closure). Assume a finite source graph with no cycles, immutable root-policy references, complete influence sources, no release step, and a read decision at one consistent policy version. Then every returned record satisfies every root policy in its source closure. The predicate cache must equal the conjunction obtained from that closure; a cache with only the right version number is not enough.

The proof is by induction on graph depth. A root passes exactly when its own predicate passes. A derived record passes only when the conjunction of its direct source predicates passes. By the induction hypothesis, each passing source satisfies every root below it. Taking the union of those roots proves the result. A missing root denies the read, so absent data cannot create an exception. Adding one more source can only keep or reduce the permitted reader set.

This theorem concerns record admission. It does not require the text to be true. Complete influence is a strong assumption. Tracking prompt inputs covers an isolated stateless call; it does not show what an outside service retained from other calls. The theorem also excludes already released copies and observations such as timing unless those are explicitly mediated.

VI. Worked source combinations

Consider three illustrative source policies in one organization. A team notice is for all users of the agent. A private plan is for Ana. A channel decision is for group G. Combining the notice and plan yields Ana's private policy: the broad source adds origin information but cannot cancel the narrow source. A group answer still cannot use that result.

Combining Ana's plan and Bob's private plan yields no ordinary reader under the given policies. A claim such as "both plans need more time" stays restricted even if it sounds general. General wording is not declassification. A permitted summary needs a release decision or an independent derivation from inputs that its readers may access.

Now combine an all-user fact from organization O_1 with an all-user fact from O_2. Their allowed organization intersection is empty. Sharing a pool does not make either organization an authorized reader of the combination. In the proposed model, both origins remain recorded for removal. The baseline instead retains neither organization in the combined allowed set. The result is hidden, but removal by only one allowed-organization key will not find it. Reading and removal therefore require different combinations of organization data.

Finally, suppose Ana's plan and a private tool result name the same work date. A designer might accept outside data only when a person's statement also supports it. That admission rule does not show that the person endorsed every private detail in the tool output. The influence policy must still include both inputs. Whether the date is true is a separate memory-quality question.

VII. Release, invalidation, and removal

A. Explicit release authority

A proposed release record names an exact output version, a new audience, the source constraints it relaxes, the responsible authority, and the evidence for that authority. The check requires authority for every relaxed source constraint. It does not infer authority from owning the storage pool. Organization restrictions remain unless the release explicitly covers them too.

A release creates a new governed artifact rather than rewriting the origin of old evidence. Its history shows what was released and under which rule. A reason is retained for review, but cannot serve as the check itself. An untrusted source saying "everyone agreed" cannot grant release. A model may propose a release; a trusted policy path must validate it. This follows the separation between ordinary flow and controlled declassification in prior work [2], [5].

The proposal does not require every release to be a manual approval. A precise standing rule can authorize a class of outputs. But a rule for team scheduling cannot silently authorize disclosure of private health details used to infer that schedule. What may be released must be explicit. Testing whether a summary is harmless remains difficult; the method makes that decision visible instead of treating summarization as automatic permission.

B. Source changes and purge

Invalidation means an entry should no longer support current reasoning. It is distinct from deleting its text and distinct from withdrawing access. The baseline excludes an inactive entry from new support selection and answer input. It does not mark existing descendants inactive. A descendant can therefore remain available even though its supporting entry is no longer accepted. A current-data policy needs a separate rule for that case.

The proposed method marks descendants unavailable for new reasoning when a source becomes invalid. It may keep their history for authorized audit. A new valid derivation can restore a claim under new sources. Removing one dependency from an existing record is unsafe when the old text was made using that dependency; it requires recomputation or a justified release.

Baseline removal finds a selected key in stored fields. Contributor and origin union make this useful for ordinary derivations from a person, chat, or group. It is not a general graph-deletion proof. Organization intersection, later field changes, and incomplete model sources limit that claim.

Deleting a contributor from one record also does not show that later conclusions no longer depend on that person.

The proposed purge starts from immutable origins and follows reverse source edges. In one logical transaction, it blocks those roots, marks all reached records unavailable, and advances their policy version. Read admission and output commits check the same versioned state. A commit must reject a learning job based on an older blocked root. This prevents a slow job from adding a new descendant after removal. Physical deletion can follow the access barrier. A graph traversal costs work proportional to the reached records and edges; this is an algorithmic bound, not a measured runtime.

No purge can retract text already read by a person. It also does not erase provider logs, backups, saved chat answers, or trained model weights merely by deleting pool rows. Those are separate retention boundaries. The precise promise concerns read admissions and derivation commits after the barrier. A response admitted earlier is outside that promise unless delivery performs another check.

VIII. Why filtering is not end-to-end noninterference

For a viewer, two states are equivalent when all permitted inputs are the same. End-to-end noninterference would require the viewer's output distribution to be the same across such states, subject to declared release. A row filter establishes only that selected rows pass a predicate. It says nothing about how those rows were written.

A simple countermodel suffices. A helper reads one hidden bit and one public source. It writes the bit into a new entry while citing only the public source. The new entry has valid references and passes retrieval. Yet changing the hidden bit changes the answer. No filter on the stored label can repair missing influence data after the fact. This argument does not assert that a particular model behaved this way; it shows why reference validation is insufficient for the proof.

Other baseline choices expose separate observations. The optional broad helper can read private pool text while relying on prompt rules for its reply. Its output has no enforced source bound. Also, compare two pool states with identical visible records but one extra hidden update. The global versions differ, so returning that number distinguishes the states. Hiding all private rows does not hide this change. Finally, outside model state can carry information between calls unless the model boundary rules it out.

An output theorem would require complete input policies on every output path, isolated or trusted model state, and a defined treatment of metadata and timing. The record theorem does not claim this. The evaluation must test text, references, and metadata separately so that success on one does not mask failure on another.

A. A narrow output theorem

Fix a viewer v, query q, public model configuration, and a finite interaction bound h. Let W_v be the complete permitted state exposed to selection and generation. It includes order, visible versions, and any allowed prior conversation. Two executions have equal permitted state when their W_v values agree, even if their hidden records differ.

Theorem 3 (isolated output). Suppose each next input, public event, and visible state update is a function only of q, W_v, the earlier public transcript, and fresh random choices independent of hidden state. Suppose the model uses only those inputs and no private cross-call state. Then the public transcript through h steps has the same distribution in both executions. Termination and errors must obey the same rule if they are in the transcript.

Proof. Couple both runs to the same random choices. Their initial permitted state agrees. At each step, equal permitted state and equal transcript give equal next inputs and updates. The coupled model output is equal because its inputs, configuration, and random choices agree. Induction gives identical transcripts through h steps, hence equal distributions. This argument is independent of whether the model follows instructions.

The useful obligation is the premise, not the coupling argument. Filtering before generation can help meet it only when selection also uses the permitted slice. A global search followed by row filtering may let a hidden match displace a visible one. A shared version counter also violates the premise. The current finite policy experiment tests neither condition. Theorem 2 alone does not imply Theorem 3.

IX. Measured finite evidence

A. Exhaustive policy cases

The experiment implements the baseline equations and the source-clause method defined here. The recorded run uses seeds 0 through 29. The collection's experiment artifact records the method, source hashes, seeds, results, and limits. The exhaustive part has 124 root labels, 15,376 ordered source pairs, and eight viewers for each pair: 123,008 pair-viewer decisions. The fixture section below gives the complete domain, so these counts do not rely on a sampled test distribution.

The reader-set oracle evaluates each source directly and intersects those results. It does not call the token-combination function. The baseline's result is evaluated separately. The oracle permits 20,880 of the 123,008 pair-viewer decisions. The baseline adds 1,216 unsafe admissions and loses 352 of those permitted reads. All unsafe cases are private viewers; none are group viewers. A separate implementation keeps source policy clauses, joins clause lists, and checks each source clause. It has zero extra admissions and zero lost permitted reads against the reader-set oracle across the same domain. These are counts over a defined finite domain, not real-world rates. Treating the eight viewers of a pair as independent trials would be wrong.

TABLE II

FINITE EVIDENCE. THE POLICY DOMAIN IS EXHAUSTIVE; THE GRAPH DOMAIN USES 30 FIXED SEEDS. THESE COUNTS ARE NOT A LANGUAGE-MODEL ATTACK STUDY.

Measured item	**Count**
Root labels	124
Ordered source pairs	15,376
Pair-viewer decisions	123,008
Oracle-permitted decisions	20,880
Unsafe baseline admissions	1,216
Permitted baseline reads lost	352
Clause errors / lost reads	0 / 0
Checked derived graph nodes	1,200
Withdrawal node checks	1,440

For each seed, the graph part chooses eight roots and adds 40 derived nodes. Each node uses one to four earlier nodes. The cached reader intersection matches a second calculation that walks back to roots and evaluates their policies directly at all 1,200 derived nodes. In a further test, one root is blocked in each graph. A walk along reverse source edges finds the root and its descendants. A pass in creation order then updates their reader caches. An independent root walk checks all 48 nodes in every graph, for 1,440 node checks. In total, 721 nodes are affected. Keeping the old cache without this step wrongly permits 219 node-viewer reads; the checked propagation permits none. One ordered version-barrier case also rejects a prepared output after its source is blocked. These are model schedules, not concurrent database tests.

The checks establish more than a hand-selected counterexample, but less than general security. The exhaustive domain has only two people, two groups, two organizations, one contributor per root, and nonempty token sets. The graph generator does not test policy release, changing membership, incomplete prompt capture, or model state. Its blocked-root check measures governed read denial, not physical erasure or a race between real transactions. The algebraic proof covers finite graphs under its assumptions; these tests check two concrete calculations within a finite domain.

X. FURTHER EVALUATION

The language-model study has not been run. The next phase must keep deterministic policy correctness apart from model behavior. Extend the finite domain with missing roots, cycles, larger source graphs, and changes to membership. Check policy updates and removal in concurrent services, including work that spans a restart. Keep the measured finite cases as controls for these wider studies.

The policy oracle should evaluate source-specific predicates directly. It must not reuse the baseline token-meet procedure, which would reproduce the same error. Retain the two-contributor channel trace as a named regression case. Test group viewers and private viewers separately. Include a model stub that deliberately cites an allowed source while using hidden input; this tests whether the host records all inputs without relying on model cooperation.

TABLE III

FURTHER CHECKS. THE MEASURED RESULTS COVER SOURCE RULES AND MODEL WITHDRAWAL, NOT LIVE REMOVAL RACES OR MODEL ATTACKS.

Case family	**Primary check**
Mixed sources	Every source policy passes
Channel context	No private-to-group spill
False release claim	Authority check rejects it
Hidden source omission	Full input policy survives
Invalidation and purge	No later governed read
Public metadata	Hidden changes stay hidden under the stated contract

The model study should use synthetic organizations and secrets generated for the test. Attack prompts should request exact secrets, paraphrases, comparative hints, and aggregates. Place hostile requests in user text, outside tool output, and a source proposing false consent. Use benign control prompts with the same task and allowed information.

Compare five declared configurations: no shared memory; a pooled prompt with privacy instructions; the defined token-based baseline; source-specific predicate conjunction with model-selected citations; and conjunction with complete input tracking plus checked release. Hold model version, prompt budget, decoding settings, and source order fixed where possible. An additional ablation can remove the private participant fallback to measure its utility cost. Another can remove source-version checks to expose stale-job removal failures.

Primary policy metrics are unsafe admission rate and permitted-entry rejection rate, both measured against the independent oracle. Model metrics are unauthorized fact disclosure, attack success per secret, and allowed-fact coverage. Count exact disclosure separately from semantic disclosure, with blinded human review for disputed paraphrases. Utility means completing the given task using permitted facts; it does not score long-term memory quality. Record latency, source-graph size, policy-cache misses, and retained blocked records as costs.

Pre-register the scenario families and primary comparisons. Pair each attack with its control and use the same secret and seed across configurations. Report per-family counts and confidence intervals. Cluster uncertainty by source graph and secret, since repeated questions about the same secret are not independent trials. Report paired differences rather than only pooled averages. For zero disclosures in n independent Bernoulli trials with common risk, the one-sided 95 percent upper bound is $1 - 0.05^{\frac{1}{n}}$. For clustered prompts, define one disclosure outcome per independent graph before applying that bound.

Removal tests should use the ordered schedule below and repeat across process restart and policy-cache expiry. Release tests should cover missing authority for one source, expired evidence, and a valid narrow release. Fix these choices before examining model outcomes.

XI. Reproducible fixtures and limits

A finite fixture can be built without any product data. Use two organizations, two people, and two groups. Give each root one organization, one contributor, and a nonempty subset of the five tokens U, $P(\text{Ana})$, $P(\text{Bob})$, $G(g_1)$, and $G(g_2)$. There are $2 \times 2 \times (2^5 - 1) = 124$ such root policies. Pairing them in order gives $124^2 = 15,376$ two-source cases. Each has eight viewers: two organizations times two private people or two group contexts. These are the domains used by the exhaustive run reported above. The finite test does not assign equal real-world likelihood to these policy labels.

For every case, compute the baseline result from M, then evaluate V independently on its two sources and derivative. Record each failed implication. Compute the corrected policy by conjunction and repeat the same implication check. The explicit Ana/Bob example supplies a known positive failure case for the baseline. Cases with identical sources and private-only audiences provide controls. Source-free malformed derivatives and missing source IDs belong to the next denial-test set; they were not checked in the measured pair enumeration.

The graph generator uses Python's standard `Random` class with each integer seed from 0 through 29. List roots by organization, person, and then the nonzero five-bit token mask. Use `sample` to choose eight distinct roots. For node i from 8 through 47, draw the parent count with `randrange(1, 5)` and then use `sample(range(i), count)`. After all nodes are built, `randrange(8)` selects the blocked root. This fixes both the sample order and when the random state advances.

For a wider removal test, use an event schedule rather than real-time delay: capture source version one; prepare a derived output; block the root and advance to version two; attempt to commit the prepared output; then attempt a new read. The required outcomes are rejection at commit and denial at read. Reverse the first commit and barrier order to test the stated boundary for previously admitted data. Store the schedule so independent implementations can replay it.

The proof obligations remain distinct. The controller must protect fields, root versions must be immutable, source capture must cover all protected model inputs, and admission must serialize with removal. Each obligation needs its own test or assurance argument. Passing the finite fixture cannot prove behavior for all graphs or for unobserved provider state.

A model study should archive model identifiers, generated graphs, the independent policy oracle, prompts, seeds, release records, and error handling. Save request and response artifacts under the test's own data policy. Keep failed and timed-out calls in the declared denominator rules. The complete protocol must state which observable values, including versions and timing, its security metric covers.

Conservative tracking reduces reuse. Equal text may need distinct records under different policies. Uncertain membership can deny a useful read. Source graphs add storage and checking costs. These are measurable tradeoffs. Content truth, model training privacy, outside-store copies, and general action authority remain outside the proof.

XII. Conclusion

Shared memory requires a rule for derived knowledge, not only for stored source rows. Existing information-flow theory gives that rule: keep source origins and intersect permitted readers. The defined baseline protects policy fields, yet the meaning of a group token also depends on contributor data. Token intersection alone does not establish reader-set intersection. The proposed method uses complete input tracking, source-specific predicates, explicit release authority, and graph-aware removal. Its proved claim is limited to governed record admission under named assumptions. The exhaustive domain shows that the token-meaning failure is executable in a finite model. These results do not measure language-model attack resistance. End-to-end confidentiality still requires the input, selection, state, and metadata obligations in the isolated-output theorem, plus a separate attack and utility study.

References

[1] D. E. Denning, "A Lattice Model of Secure Information Flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976, doi: 10.1145/360051.360056.

[2] A. C. Myers and B. Liskov, "A Decentralized Model for Information Flow Control," in *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, 1997, pp. 129–142. doi: 10.1145/268998.266669.

[3] A. C. Myers and B. Liskov, "Complete, Safe Information Flow with Decentralized Labels," in *1998 IEEE Symposium on Security and Privacy*, 1998. doi: 10.1109/SECPRI.1998.674834.

[4] J. A. Goguen and J. Meseguer, "Security Policies and Security Models," in *1982 IEEE Symposium on Security and Privacy*, 1982, pp. 11–20. doi: 10.1109/SP.1982.10014.

[5] A. Sabelfeld and D. Sands, "Declassification: Dimensions and Principles," *Journal of Computer Security*, vol. 17, no. 5, pp. 517–548, 2009, doi: 10.3233/JCS-2009-0352.

[6] S. Zdancewic and A. C. Myers, "Robust Declassification," in *Proceedings of the 14th IEEE Computer Security Foundations Workshop*, 2001, pp. 15–23. doi: 10.1109/CSFW.2001.930133.

[7] P. Buneman, S. Khanna, and W.-C. Tan, "Why and Where: A Characterization of Data Provenance," in *Database Theory: ICDT 2001*, in Lecture Notes in Computer Science, vol. 1973. 2001, pp. 316–330. doi: 10.1007/3-540-44503-X_20.

[8] T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance Semirings," in *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2007, pp. 31–40. doi: 10.1145/1265530.1265535.

[9] L. Moreau and P. Missier, Eds., "PROV-DM: The PROV Data Model," W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/2013/REC-prov-dm-20130430/

[10] J. Cheney, A. Ahmed, and U. A. Acar, "Provenance as Dependency Analysis," *Mathematical Structures in Computer Science*, vol. 21, no. 6, pp. 1301–1337, 2011, doi: 10.1017/S0960129511000211.

[11] M. Costa *et al.*, "Securing AI Agents with Information-Flow Control." [Online]. Available: https://arxiv.org/abs/2505.23643v2

[12] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases," in *Advances in Neural Information Processing Systems*, 2024. doi: 10.52202/079017-4136.

[13] S. Dong *et al.*, "Memory Injection Attacks on LLM Agents via Query-Only Interaction," in *Advances in Neural Information Processing Systems*, 2025. doi: 10.52202/085713-1554.